

Stacks & Queues

Two linear data structures that differ in the order in which they remove elements. Both appear in algorithms, operating systems, and everyday software.

1. Stack: last in, first out (LIFO)

A stack adds and removes elements at the **top**. **Push** adds an element, **pop** removes the most recently added element, and **peek** reads the top without removing it. Think of a pile of plates: the last plate placed on top is the first one removed.

Example: Push A, then B, then C. Three pops return **C, B, A**.

Uses: function calls, undo operations, balanced-parenthesis checking, and depth-first search.

2. Queue: first in, first out (FIFO)

A queue adds elements at the **rear** and removes them from the **front**. **Enqueue** adds an element; **dequeue** removes the oldest element. Think of a line at a ticket counter: the first person to join is served first.

Example: Enqueue A, then B, then C. Three dequeues return **A, B, C**.

Uses: breadth-first search, print-job queues, and first-come-first-served task processing.

Operation / property	Stack	Queue
Removal order	Newest element first	Oldest element first
Python implementation	list: <code>append()</code> , <code>pop()</code>	deque: <code>append()</code> , <code>popleft()</code>
Typical operation cost	$O(1)^*$	$O(1)$ with deque

*For a Python list stack, `append` is amortized $O(1)$; removing the last item is $O(1)$. Removing the first item with `list.pop(0)` is $O(n)$, so use `collections.deque` for a queue.

3. Try it in Python

```
from collections import deque
```

```
stack = [10, 20, 30]
print(stack.pop())      # 30: newest item
```

```
queue = deque([10, 20, 30])
print(queue.popleft())  # 10: oldest item
```

Check your understanding: Which structure suits an undo button? Which suits breadth-first search? Why is `list.pop(0)` a poor choice for a large queue?